Module 3: Web Services and APIs

3.1 REST API design and implementation

A REST API (Representational State Transfer Application Programming Interface) is a web service that allows communication between client and server applications over HTTP. REST follows a stateless client-server architecture, meaning each request must include all the information the server needs to process it. The server does not store client content between requests, which makes APIs easier to scale and maintain.

REST APIs treat everything as a resource, and each resource is identified by a URI (Uniform Resource Identifier). Clients interact with these resources through predictable endpoints, using standard HTTP methods and status codes to ensure consistency.

3.1.1 Core HTTP Methods

REST APIs rely on standard HTTP methods to perform actions on resources:

- GET – Retrieve data (e.g., /users returns all users)
- POST – Create a new resource (e.g., /users creates a user).
- PUT – Update or replace an existing resource (e.g., /users/5).
- PATCH – Partially update a resource (e.g., /users/5 with only one field).
- DELETE – Remove a resource (e.g., /users/5).

Using these methods consistently makes APIs easier to learn and integrate.

3.1.2 Data Formats

REST APIs can support multiple formats for data exchange, including JSON, XML, and YAML. Among these, JSON (JavaScript Object Notation) is the most commonly used because of its lightweight structure, simplicity, and compatibility across platforms.

3.1.3 Performance Considerations

To improve efficiency and reduce server load, REST APIs should use caching mechanism such as:

- Cache-Control headers – to specify caching policies.
- ETag headers – to allow conditional requests and avoid sending unchanged data.

REST APIs also support a layered system. Clients do not need to know if they are communicating directly with the server or though intermediaries such as load balances or proxies. This separation improves scalability and reliability.

Hypermedia (HATEOAS)

REST APIs can include HATEOAS (Hypermedia as the Engine of Application State), which provides links in responses. This allows clients to discover related resources dynamically, rather than hardcoding endpoint structures. Example:

```
{
  "id": 5,
  "name": "Alice",
  "links": {
   "self": "/users/5",
   "orders": "/users/5/orders"
  }
}
```

Steps for implementing a REST API

1. Define API Requirements
    a. Identify the APIs purpose, business needs, and target users.
2. Design Endpoints
    a. Use nouns for resource names (/products, /orders) and structure them clearly.
    b. Keep endpoint names consistent and predictable.
3. Choose Tools and Frameworks
    a. Examples: Express.js (Node.js), Django REST Framework (Python), Spring Boot (Java), or Laravel (PHP).
4. Handle Error Responses
    a. Return proper HTTP status codes (e.g., 400 Bad Request, 404 Not Found, 500 Internal Server Error).
    b. Provide clear error messages in the response body.
5. Authentication and Authorization
    a. Implement security mechanisms such as API keys, OAuth 2.0, or JWT (JSON Web Tokens).
6. Validation and Security
    a. Validate input data to prevent injection attacks and bad requests.
    b. Use HTTPS for seure communication.
7. Naming Conventions

    a. Use plural nouns for resources (e.g., /users not /user)
    b. Avoid verbs in endpoints (actions should be presented by HTTP methods).
8. Pagination for Large results
    a. Apply query parameters (e.g., /users?page=2&limit=50) for better performance and usability.
9. API Versioning
    a. Prevent breaking changes by versioning the API (e.g., /api/v1/orders).
10. Documentation
    a. Use tools like Swagger/OpenAPI or Postman to generate clear, interactive documentation.
11. Rate Limiting
    a. Limit client requests to protect against abuse and ensure fair usage.

REST API design and implementation follows a set of principles that make web services reliable, scalable and easy to use. By defining clear endpoints, using standard HTTP methods, ensuring proper authentication, validating input, and documenting the system, developers can build APIs that are both secure and developer-friendly.

3.2 SOAP web services

SOAP (Simple Object Access Protocol) is a protocol specification for exchanging structured information in the implementation of web services. Unlike REST, which is an architectural style, SOAP is a formal protocol that relies on XML for message formatting and usually uses HTTP or SMTP as the transport protocol. SOAP provides a rigid and standardized way of communication, making it widely used in enterprise-level application where security, reliability, and strict contracts are critical.

Key Characteristics of SOAP

1. Protocol-Based
    a. SOAP is a formal protocol with strict rules defined by the W3C.
    b. It defines both the structure of the messages and the way they are transmitted.

2. XML Messaging
    a. All SOAP messages are encoded in XML format.
    b. This makes SOAP language-independent and platform-independent, as XML can be parsed by any system.
3. Transport Independence
    a. Although most SOAP services run over HTTP/HTTPS, SOAP can also use SMTP, TCP, or JMS for communication.
4. Strongly Typed Contracts.
    a. SOAP services use WSDL (Web Service Description Language) to describe the operations offered, the data types involved, and how clients can consume the service.
    b. This ensures clients and servers agree on a strict communication contract.
5. Security and Reliability.
    a. SOAP supports WS-Security, which includes message encryption, digital signatures, and authentication.
    b. It also supports standards for reliable messaging, making it a preferred choice in sensitive applications like banking, healthcare, and government systems.

Structure of a SOAP Message

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header>
      <!-- Optional metadata such as authentication tokens -->
  </soap:Header>
  <soap:Body>
      <m:GetProductDetails xmlns:m="http://example.com/products">
       <m:ProductID>123</m:ProductID>
      </m:GetProductDetails>
  </soap:Body>
</soap:Envelope>
```

- Envelope – the root element that defines the XML document as a SOAP message.
- Header – Optional section containing metadata such as authentication or transaction details.
- Body – Contains the actual request or response data.

- Fault (Error Handling) – if an error occurs, a <soap:Fault> element is returned in the body.

SOAP vs REST

Feature	SOAP	REST
Type	Protocol (strict rules)	Architectural Style (flexible)
Message Format	XML only	JSON, XML, YAML, etc.
Transport	HTTP, SMTP, TCP, JMS	HTTP only
Contract	Strongly typed via WSDL	Not required, but can be use OpenAPI docs
Security	WS-Security (encryption, signatures)	HTTPS + OAuth, JWT
Use Cases	Banking, payment processing, healthcare, enterprise apps	Web apps, mobile apps, lightweight APIs

Steps for implementing SOAP Web Services

1. Define Service Requirements
    a. Identify the operations (methods) the service should provide and the data to be exchanged.
2. Create the WSDL
    a. Write the Web Services Description Language document, which defines:
        i. Service endpoint
        ii. Operations available
        iii. Input/Output parameters and data types
3. Develop the Service
    a. Implement the service using server-side technologies such as:
        i. Java (JAX-WS, Apache CXF)
        ii. .NET (WCF)
        iii. PHP (SOAP extension)
4. Deploy the Service

    a. Host the service on a web server or application server so clients can access it via the service endpoint (usually an HTTP URL).
5. Client Stub Generation
    a. Use tools (e.g., wsimport in Java, WCF in .NET) to generate client-side code from the WSDL.
    b. This ensures clients communicate correctly with the service.
6. Error Handling
    a. Implement proper fault responses using the <soap:Fault> element for error messages.
7. Security
    a. Configure WS-Security for authentication, encryption, and digital signatures where needed.

Example SOAP Request/Response

Request:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
      <GetOrderStatus xmlns="http://example.com/orders">
      <OrderID>567</OrderID>
      </GetOrderStatus>
  </soap:Body>
</soap:Envelope>
```

Response:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
      <GetOrderStatusResponse xmlns="http://example.com/orders">
      <OrderID>567</OrderID>
      <Status>Shipped</Status>
      </GetOrderStatusResponse>
  </soap:Body>
</soap:Envelope>
```

Advantages of SOAP

- Platform and language independent
- Strict contracts ensure interoperability between systems.

- Strong support for security and reliable messaging.
- Suitable for complex, mission-critical enterprise applications

Disadvantages of SOAP

- More verbose compared to REST due to XML formatting
- Higher learning curve and complexity
- Slower performance for lightweight applications
- Less flexible than REST for modern web and mobile apps.

SOAP web services provide a standardized and robust way of exchanging structured data between systems. With strict contracts defined by WSDL, transport independence, and strong support for security, SOAP remains a reliable choice for enterprise environments. while modern application often prefer REST due to its simplicity and speed, SOAP is still widely used in industries requiring high security, reliability, and interoperability.

3.3 API Documentation

API documentation is a critical part of the software development lifecycle. It acts as a bridge between the API provider and the developer community, offering a detailed guide on how to understand, use, and integrate the API effectively. Good documentation reduces the learning curve, shortens development time, and improves overall developer experience. Without clear documentation, even the most powerful APIs can become difficult to adopt.

Purpose of API documentation

- Guide for developers: provides step-by-step instructions on how to access and use the API.
- Reference Material: Serves as a quick lookup for endpoint details, parameters, and error codes.
- Integration Support: Helps developers integrate the API correctly into their applications.
- Consistency: Ensures all teams (internal and external) follow the same usage patterns.
- Efficiency: Reduces unnecessary support requests and saves time for both developers and API maintainers.

Components of API Documentation

A well-documented API typically contains the following components:

1. Overview
    a. A brief introduction explaining the purpose of the API, it's main features, and potential use cases.
2. Authentication
    a. Details of how users authenticate, such as API keys, OAuth 2.0 tokens, or JWT (JSON Web Tokens).
    b. Example: Authorization: Bearer <access_token>
3. Base URL
    a. The root URL for all API endpoints.
    b. Example: https://api.example.com/v1/
4. Endpoints
    a. A complete list of API endpoints, grouped by resource (e.g., /users, /orders, /products).
    b. For each endpoint, include the HTTP method, description, and URL path.
5. Parameters
    a. Detailed explanation of all parameters:
        i. Path parameters (e.g., /users/{id})
        ii. Query parameters (e.g., ?page=2&limit=50)
        iii. Body Parameters (for POST/PUT requests)
        iv. Header parameters (e.g., Content-Type: application/json)
6. Request Examples:
    a. Example requests in different formats (e.g., cURL, Python, JavaScript).
    b. Example:
7. Response Examples
    a. Sample success and failure responses in the expected format (JSON, XML).
    b. Example:
8. Error Codes
    a. A table of possible errors, including HTTP status codes and descriptions.
9. Rate limiting
    a. Information on request limits (e.g., 100 requests per minte)
    b. Explain headers used for rate-limit tracking (e.g., X-RateLimit-Limit).
10. Versioning

    a. Indicate to official SDKs or client libraries in popular programming languages (Java, Python, JavaScript, PHP)

11. Interactive Tools
    a. Tools like Swagger UI, Postman Collections, or API consoles to allow developers to experiment with endpoints in real time.

Best Practices for API Documentation

- Be clear and concise: Use plain, unambiguous language. Avoid jargon when possible.
- Maintain Consistency: Ensure parameter names, response formats, and error structures follow consistent patterns.
- Include Code Examples: Provide request/response samples in multiple languages to reach a broader audience.
- Use OpenAPI/Swagger: Adopt machine-readable formats like OpenAPI specification (OAS) to auto-generate documentation and SDKs.
- Offer Interactive Documentation: Provide live testing tools (Swagger UI, Redoc, Postman).
- Document Edge Cases: Clearly describe error handling, rate limiting, and unusual scenarios.
- Update Regularly: Keep documentation synchronized with the API version to avoid outdated references.

Well-written API documentation is just as important as the API itself. It provides developers with the necessary information to understand authentication, endpoints, parameters, error handling, and usage limits. By including examples, adopting standards such as OpenAPI, and offering interactive tools, API providers can ensure a smooth integration and encourage adoption by the developer community.

3.4 Testing and Debugging web services

Testing and debugging are critical phases in the development of web services. They ensure that the service functions as intended, handles different types of input correctly, and maintains reliability under various conditions. A web service that has not been thoroughly tested can lead to unexpected failures, security vulnerabilities, or poor performance in production. Proper testing also simplifies debugging, allowing developers to identify, trace and fix issues efficiently.

Importance of Testing Web Services

- Reliability: verifies that endpoints consistently return the expected results.
- Security: Ensures the service is protected against vulnerabilities such as injection attacks, unauthorized access, and data leaks.
- Performance: Validates that the service can handle concurrent users, high request volumes, and large payloads.
- Interoperability: Confirms that the service works across different platforms, devices, and clients.
- Maintainability: Simplifies future development by catching regressions early.

Types of Web Service Testing

1. Unit Testing
    a. Focuses on testing individual service functions or methods in isolation
    b. Example: Testing a function that validates input data before saving to a database.
2. Integration Testing
    a. Ensures different components of the system (e.g., service, database, external APIs) work together correctly.
    b. Example: Testing a payment service that communicates with both the order system and a third-party payment gateway.
3. Functional Testing
    a. Verifies that the web service meets the specified requirements.
    b. Example: Testing the GET /users endpoint to confirm it returns a list of users in the correct format.
4. Performance and Load Testing
    a. Measuring how the service performs under stress, such as concurrent requests or high data loads.
    b. Tools like Apache JMeter, Locust, or k6 are commonly used.
5. Security Testing
    a. Identifies vulnerabilities like SQL injection, cross-site scripting (XSS), or broken authentication.
    b. Example: Attempting unauthorized access to confirm the API enforces proper authentication.
6. Regression Testing
    a. Ensures that changes to the codebase do not break existing functionality.

    b. Automated test suites are useful for this purpose.

Tools for testing web services

- Postman – widely used for manual and automated API testing with support for scripting.
- SoapUI – Popular tool for SOAP and REST testing, supporting functional, load, and security tests.
- Swagger/OpenAPI UI - Allows developers to test endpoints interactively from the documentation.
- Junit/PyTest/Mocha – Frameworks for writing automated unit and integration tests in Java, Python, and JavaScript.
- Apache JMeter – For load and performance testing.

Debugging Web Services

Debugging involves identifying the root cause of issues and applying fixes. Common problems include incorrect request/response fromats, authentication errors, invalid parameters, and server-side exceptions.

Common Debugging Techniques

1. Logging
    a. Implement structured logging on both the client and server
    b. Capture request details (headers, parameters, payload) and response details (status, code, body)
2. Use of Debugging Tools
    a. Tools like Fiddle, Wireshark, or browser developer tools can capture and analyze HTTP requests and responses.
3. Check Error Responses
    a. Examine HTTP status code and error messages returned by the API.
    b. Ensure descriptive errors are returned (e.g., 400 Bad Request: Missing parameter ‘id’).
4. Isolate Components
    a. Test individual service layers (e.g., database queries, authentication module) to identify where the issue occurs.
5. Validate Data Formats
    a. Confirm that request and response payloads follow the expected schema (e.g., correct JSON fields).
6. Use Mock Servers

a. Simulate parts of the system (e.g., third-party services) to test how the API handles responses without relying on external dependencies.

Best Practices for Testing and Debugging

- Automate testing where possible to reduce manual effort.
- Cover both normal and edge cases (e.g., invalid inputs, empty payloads).
- Ensure tests run as part of a Continuous Integration (CI) pipeline.
- Use descriptive error messages to aid debugging.
- Log sufficient information without exposing sensitive data.
- Test under real-world conditions (network latency, high load).
- Document known issues and troubleshooting steps in the API documentation.

Testing and debugging web services are essential for delivering reliable, secure, and high-performing APIs. By applying different levels of testing—unit, integration, functional, performance, and security—developers can ensure comprehensive coverage. Debugging techniques such as logging, isolating components, and validating data formats help resolve issues quickly. Together, these practices build confidence in the web service, reduce downtime, and improve the developer and end-user experience.

Activity

Activity A: REST API Design (20 points)

Scenario

You are tasked to design a REST API for a **Student Management System**.

Requirements

Design endpoints for:

- Students
- Courses
- Enrollments

Tasks

1. Define **at least 5 endpoints**, including:
    - URI
    - HTTP method
    - Description
2. Include:
    - Pagination example
    - API versioning
3. Identify one authentication method to be used and explain why.

Activity B: REST vs SOAP Analysis (15 points)

Instructions

Given the following scenarios, identify whether **REST or SOAP** is more appropriate and justify your answer.

Scenario	Choice	Justification
Mobile banking app	REST / SOAP	
Government tax system	REST / SOAP	
Internal microservices	REST / SOAP	

Activity C: API Documentation Outline (15 points)

Create a **mini API documentation outline** for one REST endpoint.

Must include:

- Overview
- Endpoint and HTTP method
- Request example

- Response example
- Error codes
- Rate limiting

(May be written or diagrammed)

Activity D: Testing and Debugging Plan (10 points)

For the same API:

1. Identify **three types of testing** to be performed.
2. Recommend **one tool per testing type**.
3. Describe **one debugging technique** to be used if the API fails.

Assessment Rubric (OBE–PICAB Aligned)

Criteria	Excellent (5)	Good (3)	Needs Improvement (1)
Technical Accuracy	Correct and detailed	Minor issues	Major errors
API Design Quality	Clear, RESTful, scalable	Mostly correct	Poor design
Documentation Clarity	Complete and understandable	Adequate	Incomplete
Testing Strategy	Comprehensive and realistic	Basic	Weak
Critical Thinking	Strong justification	Some reasoning	Minimal

Lab Activity

Problem Scenario

You are tasked to create a REST API for managing **students** in a university system.
The API must allow clients (web or mobile apps) to create, retrieve, update, and delete student records.

Part A: Environment Setup

1. Create Django Project and App

```
django-admin startproject student_api
cd student_api
python manage.py startapp students
```

2. Install Django REST Framework

```
pip install djangorestframework
```

3. Update `settings.py`

```
INSTALLED_APPS = [
    ...
    'rest_framework',
    'students',
]
```

Part B: Model Design

Create Student Model (`students/models.py`)

```
from django.db import models

class Student(models.Model):
    student_id = models.CharField(max_length=20, unique=True)
    full_name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    course = models.CharField(max_length=100)
    year_level = models.IntegerField()

    def __str__(self):
    return self.full_name
python manage.py makemigrations
python manage.py migrate
```

Part C: Serializer

Create Serializer (`students/serializers.py`)

```
from rest_framework import serializers
```

```
from .models import Student

class StudentSerializer(serializers.ModelSerializer):
	class Meta:
	model = Student
	fields = '__all__'
```

Part D: Views (REST API Endpoints)

Create API Views (`students/views.py`)

```
from rest_framework import viewsets
from .models import Student
from .serializers import StudentSerializer

class StudentViewSet(viewsets.ModelViewSet):
	queryset = Student.objects.all()
	serializer_class = StudentSerializer
```

Part E: URL Routing

Configure URLs (`students/urls.py`)

```
from rest_framework.routers import DefaultRouter
from .views import StudentViewSet

router = DefaultRouter()
router.register(r'students', StudentViewSet)

urlpatterns = router.urls
```

Include in Main URLs (`student_api/urls.py`)

```
from django.urls import path, include

urlpatterns = [
	path('api/', include('students.urls')),
]
```

Part F: Testing the API (Postman)

Method	Endpoint	Description
GET	`/api/students/`	Retrieve all students
POST	`/api/students/`	Create new student
PUT	`/api/students/{id}/`	Update student
DELETE	`/api/students/{id}/`	Delete student

Sample JSON Request

```
{
  "student_id": "2024-001",
  "full_name": "Juan Dela Cruz",
  "email": "juan@student.edu",
  "course": "BSIT",
  "year_level": 3
}
```

Lab Output

- Working REST API
- Screenshot of Postman test results
- Source code submission

LAB ACTIVITY 2: Authentication, API Documentation, and Testing

Lab Title

Securing, Documenting, and Testing REST APIs in Django

Lab Duration

2–3 hours

Aligned Learning Outcomes

After this lab, students will be able to:

- Secure REST APIs using JWT authentication
- Generate API documentation using Swagger/OpenAPI
- Perform functional and security testing
- Apply debugging techniques using logs and error responses

Part A: JWT Authentication

Install JWT Package

```
pip install djangorestframework-simplejwt
```

Update `settings.py`

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
    'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
}
```

Add Token URLs (`student_api/urls.py`)

```
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)

urlpatterns += [
    path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
]
```

Part B: Protect API Endpoints

Modify ViewSet (`students/views.py`)

```
from rest_framework.permissions import IsAuthenticated

class StudentViewSet(viewsets.ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
```

```
    permission_classes = [IsAuthenticated]
```

Part C: API Documentation (Swagger)

Install drf-yasg

pip install drf-yasg

Configure Swagger (`student_api/urls.py`)

```
from drf_yasg.views import get_schema_view
from drf_yasg import openapi
from rest_framework.permissions import AllowAny

schema_view = get_schema_view(
    openapi.Info(
    title="Student API",
    default_version='v1',
    description="API documentation for Student Management System",
    ),
    public=True,
    permission_classes=(AllowAny,),
)

urlpatterns += [
    path('swagger/', schema_view.with_ui('swagger', cache_timeout=0)),
]
```

Access documentation at:
`http://localhost:8000/swagger/`

Part D: Testing and Debugging

1. Functional Testing

- Test authenticated requests using Bearer Token in Postman
- Verify correct HTTP status codes (200, 201, 401, 404)

2. Security Testing

- Try accessing endpoints without a token
- Observe `401 Unauthorized` response

3. Debugging Using Logs

```
import logging
logger = logging.getLogger(__name__)

logger.info("Student list accessed")
```

Lab Output

- Secured REST API
- Swagger UI documentation
- Screenshots of:
  - JWT authentication
  - Authorized vs unauthorized requests
- Short reflection (5–7 sentences)